



Γειά 안녕하세요 olá hello
Hej Oii Olá! ciao hola
aloha guten tag
今日は goddag Chào ahn / Chào chị
こんにちは shalom
hallo Привет

CMPU3010: Databases

- TU856
- Dr. Deirdre Lawless
 - e: deirdre.lawless@tudublin.ie



Yokoso Willkommen
Benvenuto Welkom
Willkommen
Benvindo Haere Mai
Degemer Akwaba
Dobro dosli
Tervetuloa Accoje
Hosgeldiniz
Nayak
Chào mừng
Vithaykari
Fäilte
Swagata
Benvenuti
Welkom
Witaj
Bienvenue
Aloha
Huan Ting
Salve

Contents



What is Databases 2 about?



Indicative Syllabus



Learning Outcomes



Assessments



Reading List

What do you know from Databases 1?

You have already worked with:

- relational database design;
- ER diagrams;
- primary and foreign keys;
- integrity constraints;
- normalization;
- DDL and DML;
- SELECT, joins and subqueries;
- aggregation;
- simple transactions using COMMIT and ROLLBACK.

We will use these skills rather than teach them again.



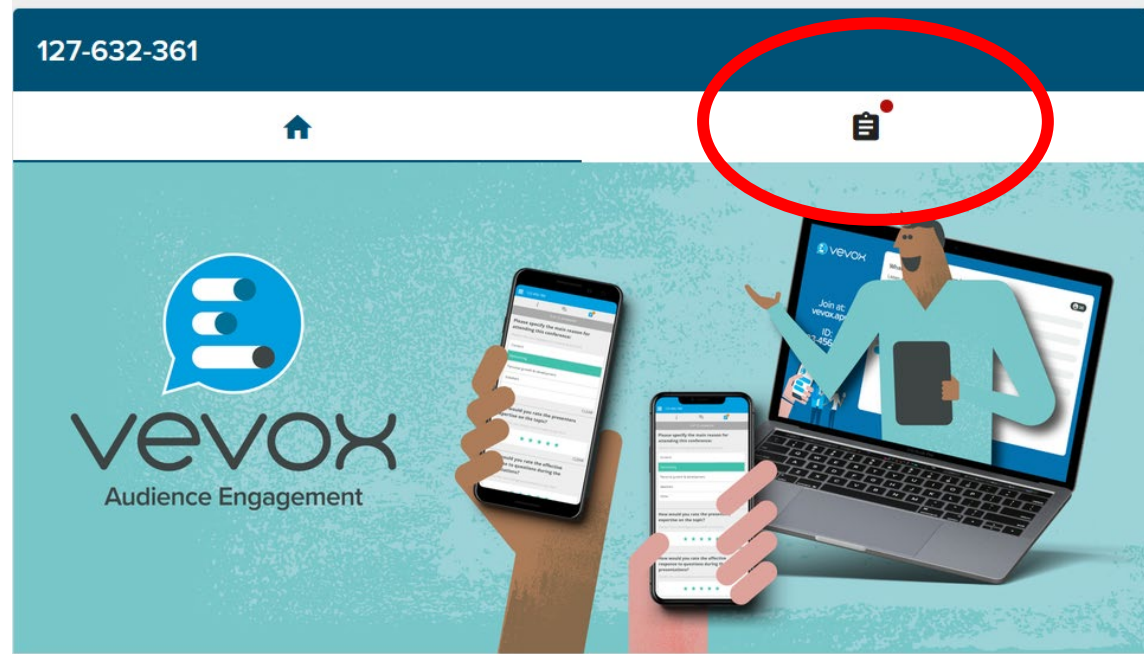
Join at:
vevox.app

ID:
127-632-361



How comfortable are you
with your knowledge and
skills?

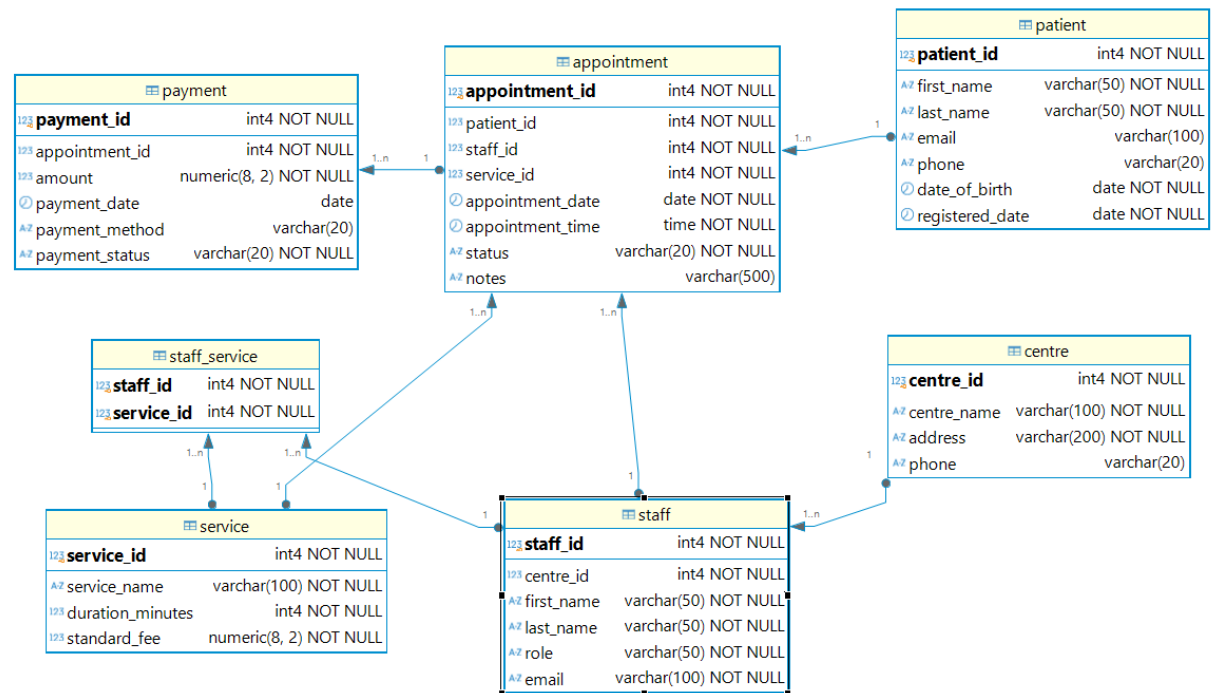
<https://vevox.app/#/m/127632361/survey/95d332aa-9ee1-4083-ba2e-d67200019114>



Our Case Study- CommunityCare

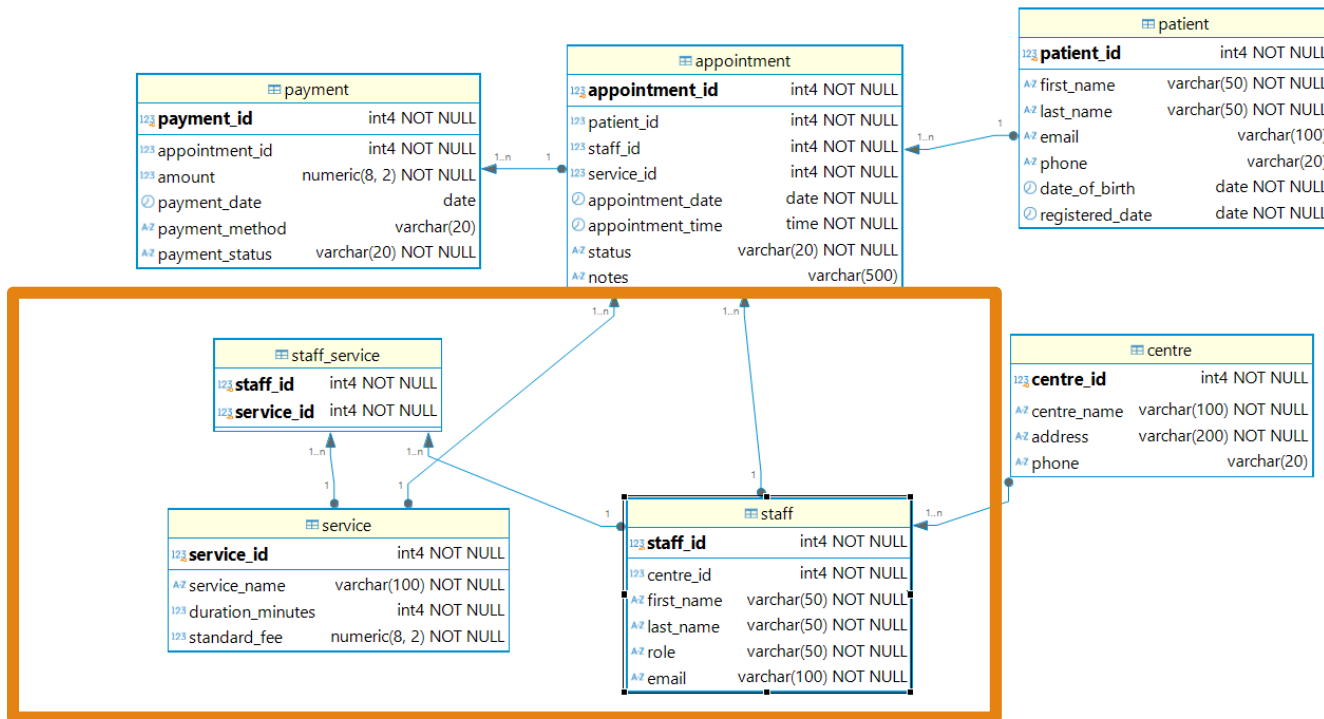
CommunityCare is a fictional community healthcare organisation providing services such as physiotherapy, dietetics, counselling and general consultations across several health centres.

Throughout Databases 2, we will use CommunityCare as a **continuing case study**.



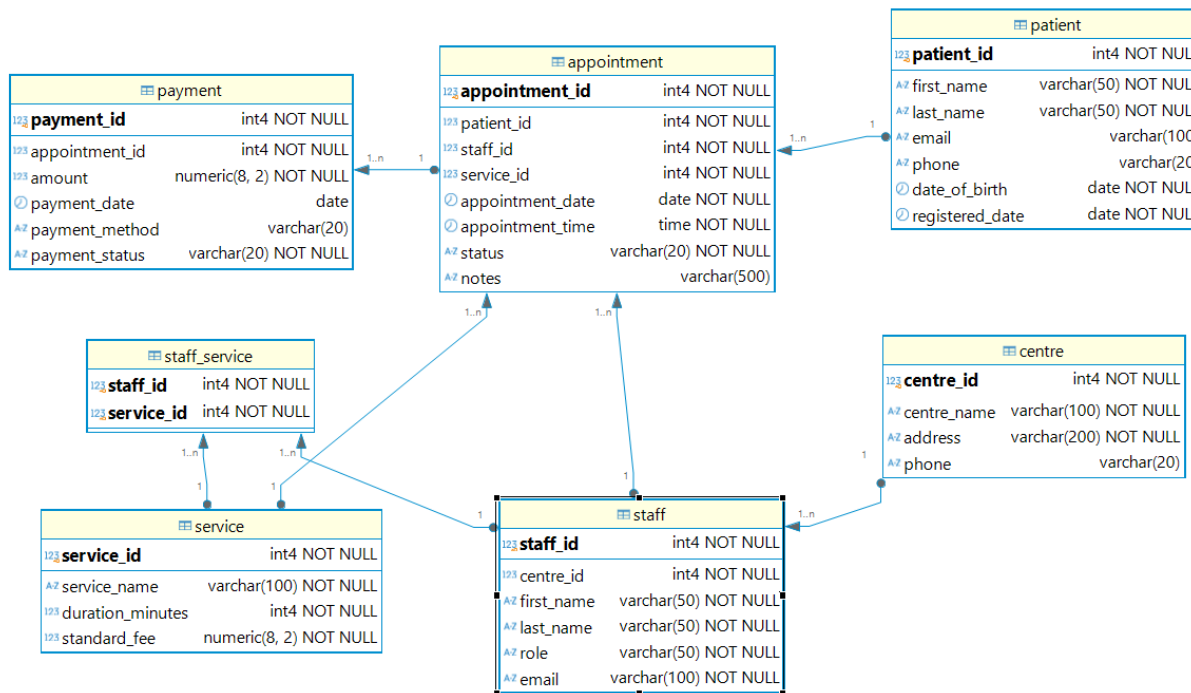
From the Lab

Why does **staff_service** need to exist?



From the Lab

What prevents two appointments for the same staff member at the same start time?



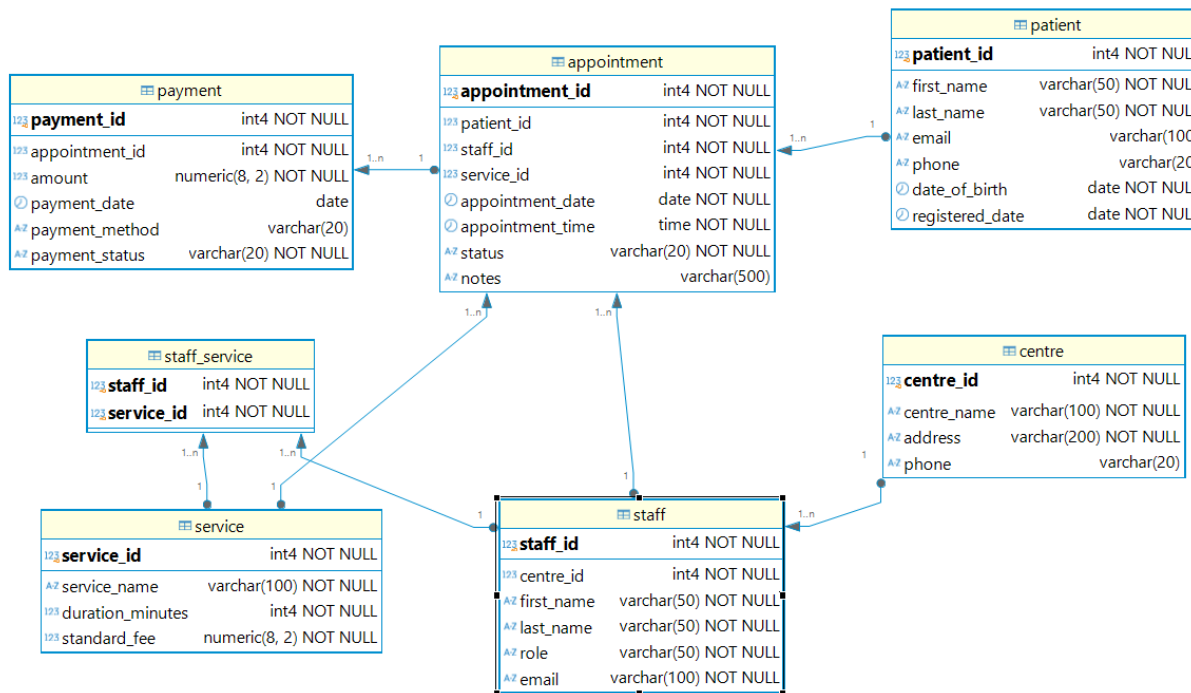
The appointment table has a **UNIQUE** constraint on:

UNIQUE (staff_id, appointment_date, appointment_time)

This means PostgreSQL will not allow two rows with the same combination of:
staff member + date + start time

From the Lab

Does it prevent OVERLAPPING appointments? E.g. a 10:00 – 11:00 appointment and a 10:30 to 11:30 appointment with the same staff member.

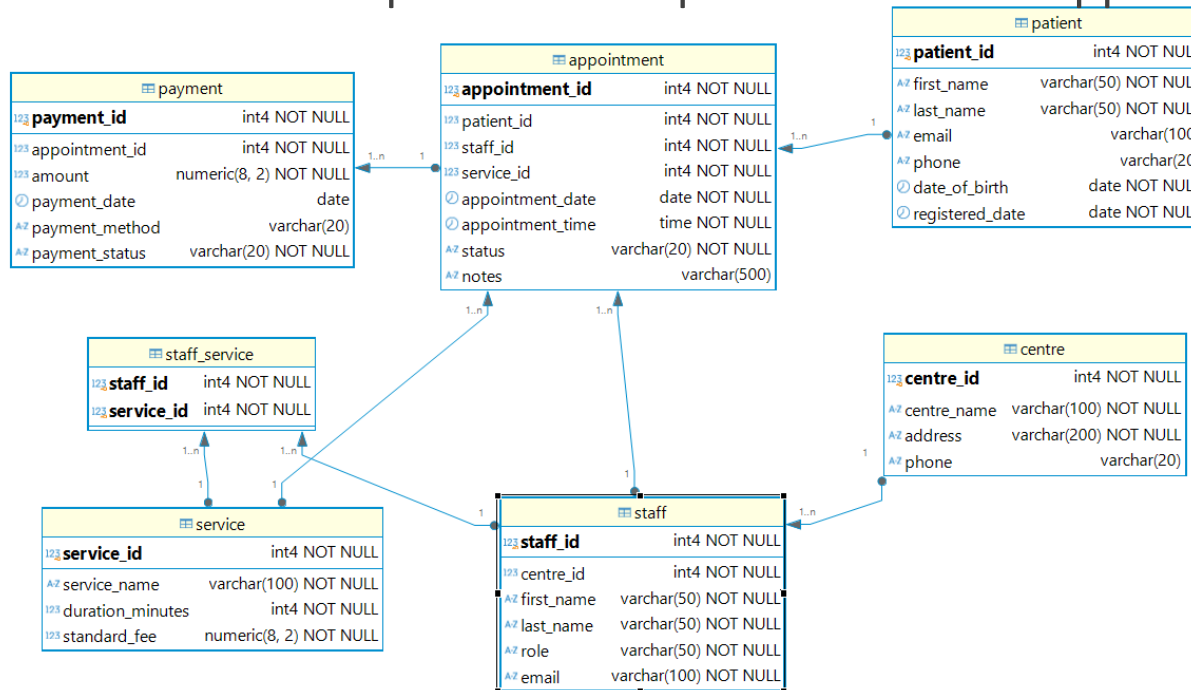


No.

We need more sophisticated logic to achieve this.

From the Lab

Why is a **LEFT JOIN** required to find patients without appointments?



Key idea: LEFT JOIN lets us find the **absence of a relationship** — in this case, patients with no appointments.

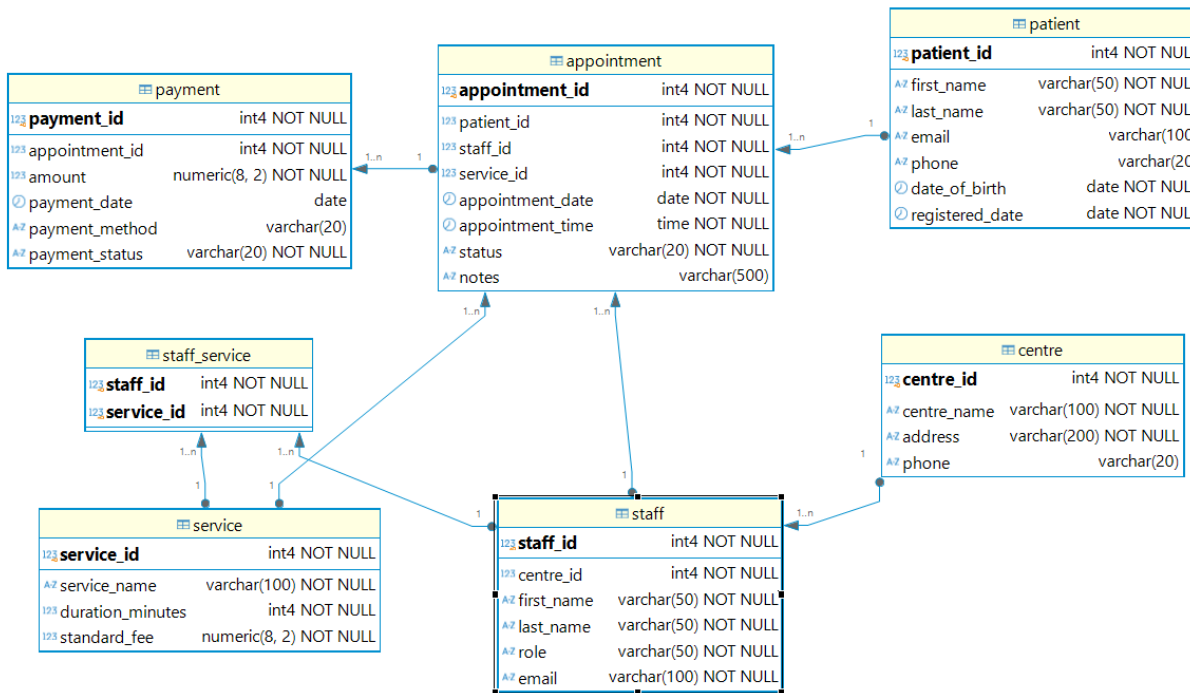
LEFT JOIN keeps **every patient**, even if they have no matching row in appointment.

For a patient with no appointment, the appointment columns are returned as **NULL**.

We then select just those patients

From the Lab

What prevents invalid appointment statuses?



Permitted values are:

Scheduled
Completed
Cancelled
No Show

They are enforced by a **CHECK** constraint.

CommunityCare Database

By the end of **Databases 1**, this might have been considered a successful database.

Then reveal:

- ✓ Normalised
- ✓ Primary and foreign keys
- ✓ Constraints
- ✓ SQL queries work
- ✓ Data can be inserted/updated

BUT...

- Can we query it efficiently?
- Can many people use it safely?
- Can we automate complex operations?
- Can we control who sees what?
- What happens when the data doesn't fit the model?

That's Databases 2.

What is the difference between Databases 1 and Databases 2?

Databases 2 asks harder questions:

- What if a query is **difficult to express**?
- What happens when **many users access the database at the same time**?
- What if two users try to change the **same data**?
- What if a query works but is **very slow**?
- Should every user have access to **every part of the database**?
- Can we **program operations inside PostgreSQL**?
- How can we use **Python to access, process and analyse database data**?
- What if some of our data **doesn't fit naturally into relational tables**?
- When might a **document database** be a better choice?

PostgreSQL + MongoDB + CommunityCare

Task C1

For each patient, show each appointment and the date of that patient's previous appointment.

How did you solve it?

Using a Correlated Subquery:

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    (
        SELECT
            MAX(a2.appointment_date)
            FROM appointment a2
            WHERE a2.patient_id =
a.patient_id
            AND a2.appointment_date <
a.appointment_date
        ) AS previous_appointment
FROM appointment a
JOIN patient p
    ON p.patient_id = a.patient_id
ORDER BY
    p.patient_id,
    a.appointment_date;
```

This works.

But SQL has tools designed for this kind of problem.

Task C1 – Using a Window Function

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    LAG(a.appointment_date)
OVER (
    PARTITION BY
a.patient_id
    ORDER BY
a.appointment_date
) AS previous_appointment
FROM appointment a
JOIN patient p
    ON p.patient_id =
a.patient_id
ORDER BY
    p.patient_id,
    a.appointment_date; This works.
```

LAG() is a **window function**.

It allows a query to access a value from a **previous row** without having to use a self-join or correlated subquery.

Task C1 – Using a Window Function

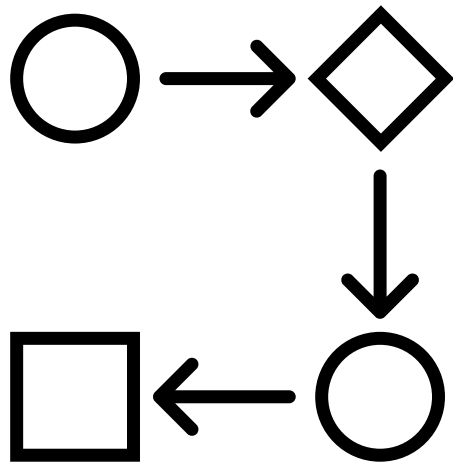
```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    a.appointment_date,
    LAG(a.appointment_date)
    OVER (
        PARTITION BY a.patient_id
        ORDER BY a.appointment_date
    ) AS previous_appointment
FROM appointment a
JOIN patient p
    ON p.patient_id = a.patient_id
ORDER BY
    p.patient_id,
    a.appointment_date;
```

How LAG() Works

- **LAG(appointment_date)** — returns the appointment date from the **previous row**.
- **PARTITION BY patient_id** — considers each patient's appointments separately.
- **ORDER BY appointment_date** — puts each patient's appointments in chronological order.
- **OVER(...)** — defines the group and ordering that LAG() operates over.

Result: each appointment can be displayed alongside that patient's previous appointment date.

Using Window Functions



Some other questions you will eventually answer about CommunityCare:

- previous/next appointment;
- rank services by popularity;
- running appointment totals;
- busiest service within each centre;
- compare this month with last month.
- and more...

Lab Part D: Transactions and concurrency



Consider two receptionists trying to book the same final appointment slot at the same time.

Receptionist A checks that 10:00 is available. Before A completes the booking, Receptionist B checks the same slot and also sees it as available. Both then attempt to book it.

D1. What could go wrong?

Lab Part D: Transactions and concurrency



Receptionist A

Checks 10:00

AVAILABLE

Books 10:00

Receptionist B

Checks 10:00

AVAILABLE

Books 10:00

What should happen?

Lab Part D: Transactions and concurrency

D2. Is BEGIN / COMMIT enough?

No.

Simply writing:

- BEGIN;
- -- check availability
- -- insert booking
- COMMIT;

does not by itself prevent another concurrent transaction from checking the same data between the check and the insert.

The eventual solution may involve appropriate constraints plus transaction **isolation/locking** and correct error handling.



Lab Part D: Transactions and concurrency



Databases 1 introduced BEGIN, COMMIT and ROLLBACK.

Databases 2 looks more closely at transactions — particularly what happens when **multiple transactions execute at the same time**.

Transactions should satisfy the **ACID properties**:

- **Atomicity** — all or nothing.
- **Consistency** — the database moves from one valid state to another.
- **Isolation** — concurrent transactions do not interfere incorrectly.
- **Durability** — committed changes persist. #

We'll study this properly - including lost updates, isolation and locking

Lab Part E: Lab E: Does everything belong in tables?

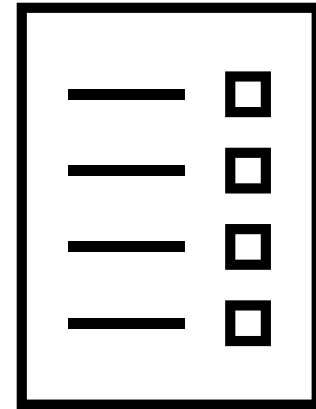
CommunityCare wants patients to complete a pre-appointment questionnaire. The questions differ substantially between services.

A physiotherapy questionnaire might record:

- pain level;
- several pain areas;
- duration;
- previous treatment.

A dietitian questionnaire might record:

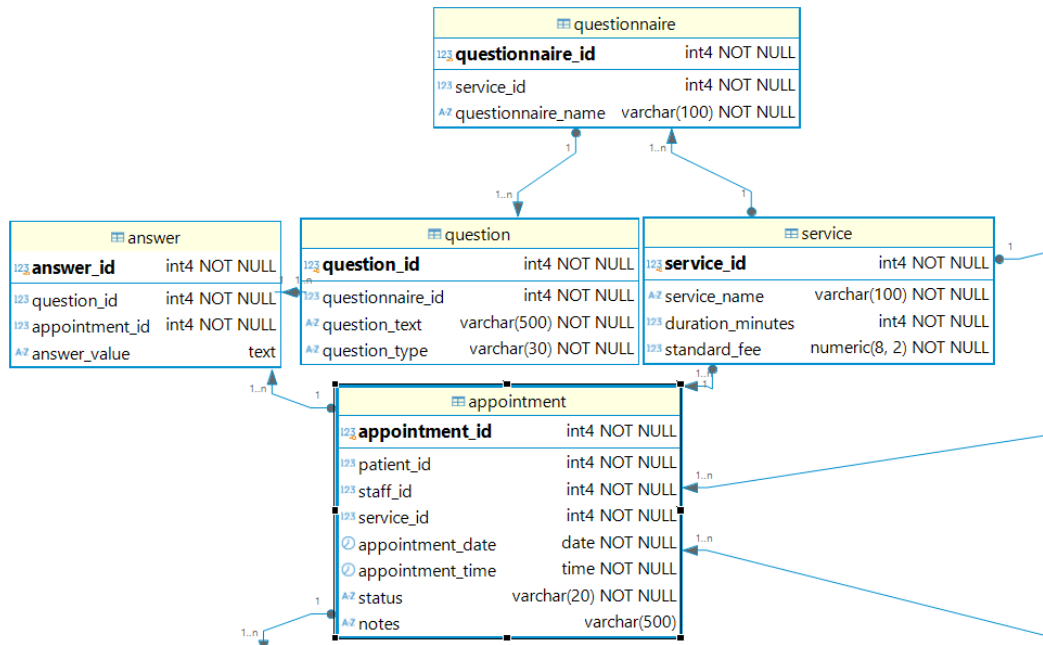
- allergies;
- dietary requirements;
- goals;
- a food diary.



Task E1. How might you store this information in the current relational database?

E1. How could questionnaires be stored relationally?

Lab Part E: Lab E: Possible Relational Solution



QUESTIONNAIRE

questionnaire_id PK
service_id FK
questionnaire_name

QUESTION

question_id PK
questionnaire_id FK
question_text
question_type

RESPONSE

response_id PK
appointment_id FK
question_id FK
response_value

The relational solution works — but is it the most natural representation?

Another way to represent the same information

We can use a **document database**. A **document** can contain fields, nested structures and arrays.

```
{  
  "appointmentId": 18472,  
  "service": "Physiotherapy",  
  "responses": {  
    "painLevel": 7,  
    "painAreas": ["knee", "lower leg"],  
    "duration": "3 months",  
    "previousTreatment": true  
  }  
}
```

Which representation is better? Relational or Document?

Which representation is better?

Which representation is better? Relational or Document?

It depends on the data and what we need to do with it.

This is a key aspect of the module that we are going to explore.



How efficient are your queries?

A correct query isn't necessarily an efficient query.

Lets look at one of our simple queries:

- `SELECT *`
- `FROM appointment`
- `WHERE patient_id = 1024;`

What does the DBMS do if appointment contains **50 rows**?

What if it contains **5,000,000 rows**?

How can we find out?

How efficient are your queries?

```
EXPLAIN ANALYZE
SELECT *
FROM appointment
WHERE patient_id = 1024;
```

EXPLAIN ANALYZE executes the query and shows how PostgreSQL processed it.

It can show:

- sequential scan vs index scan;
- join strategies;
- rows processed;
- execution time;
- expensive parts of the plan.

Correct SQL isn't necessarily efficient SQL.

Who should be allowed to do what in the CommunityCare Database?

Receptionist

- Create and manage **appointments**.
- View and update **patient contact details**.
- Should **not** see clinical information.
- Does not need access to management reports.

Practitioner

- View their **own appointments**.
- Access relevant patient details for those appointments.
- Create and update **clinical information** for their patients.
- Does not need access to management reports.

Manager

- View **summary appointment information** and service activity.
- Limited access to patient information.
- Should not normally access detailed clinical information.
- Access **reports and aggregated data** for planning and management.

Database Administrator (DBA)

- Responsible for the **database system itself** rather than delivering healthcare services.
- Creates and manages **users, roles and permissions**.
- Maintains database objects and configuration.
- Manages security, backup/recovery and performance.
- Has extensive technical privileges, but these should be used **only when required for administration**.

Should everyone be connecting to the database with the full privileges of the postgres user account?

Who should be allowed to do what in the CommunityCare Database?

Different Users — Different Access Needs

A multi-user database should not give every user access to everything.

Permissions should reflect what each user needs to do their job.

User	Appointments	Patient contact	Clinical information	Reports
Receptionist	✓	✓	X	X
Practitioner	Own	Relevant	✓	X
Manager	Summary	Limited	X	✓
DBA	✓	✓	✓	✓

Key principle: give users only the access they need — the **principle of least privilege**.

We will look at how we can get PostgreSQL to enforce the rules.

Databases Don't Operate in Isolation

We will also access CommunityCare from **Python**.

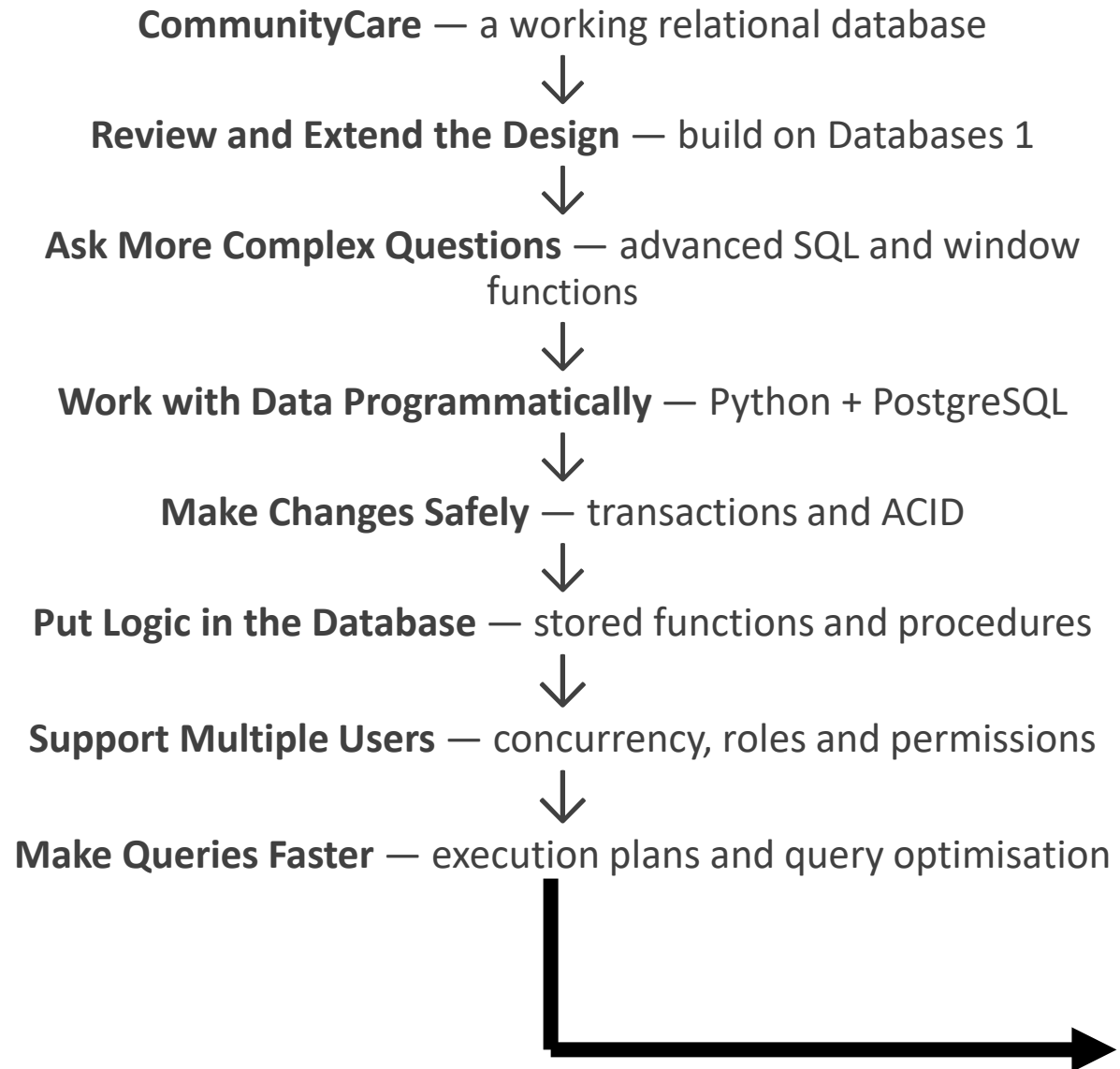
```
with conn.cursor() as cur:
    cur.execute("""
        SELECT service_id, COUNT(*)
        FROM communitycare.appointment
        GROUP BY service_id
    """)
    rows = cur.fetchall()
```

Python will allow us to:

- connect to PostgreSQL and MongoDB;
- execute queries and transactions;
- retrieve database data;
- process and analyse results;
- combine database programming with pandas.

Where are
we going?

One case study.
Increasingly
complex
problems.
Different database
techniques to
solve them.



Where are
we going?

One case study.
Increasingly
complex
problems.
Different database
techniques to
solve them.

Question the Relational Model — where does it become awkward?



Explore NoSQL — BASE, CAP and document databases



Redesign for MongoDB — document modelling and migration



Work with Documents — CRUD and aggregation



Work with MongoDB from Python — CRUD, aggregation and analysis



Bring It Together — PostgreSQL + MongoDB

Module Overview

The material covered will build on students' knowledge and skills to include design and transaction processing in a multi-user environment.

It is suitable for students who have a strong grounding in the use of SQL and have designed relational databases using ERDs.

The student will be introduced to using a procedural language to communicate with a relational database.

In addition to this, students will migrate data to a NoSQL database and write create, read, update and delete operations for it.



Indicative Syllabus

Revision of SQL

Techniques of Data Normalization

Architecture and layering of transactions in a relational database

Concepts of procedural programming as applied in an RDBMS

Ethical and efficient choice of data storage mechanism

Allocating privileges to schema users, depending on the user requirements

ACID and BASE properties and the CAP theorem

Data modelling considerations for an efficient NoSQL database

Data storage, manipulation and retrieval using NoSQL

Preparing unstructured data for use in data analytics and visualization

Learning Outcomes

MLO1: Design databases using **ER modelling** and work with **NoSQL databases**.

MLO2: Explain and apply **ACID and BASE** principles.

MLO3: Design and implement **transactions** using stored functions, procedures, external programming languages and **REST APIs**.

MLO4: Work with **structured and unstructured data**, including combining both.

MLO5: Configure **permissions** and support different users accessing a database **concurrently**.

MLO6: Write **advanced SQL**, including window functions and advanced aggregates.

MLO7: Examine how queries are executed and **improve query performance**.

Assessment

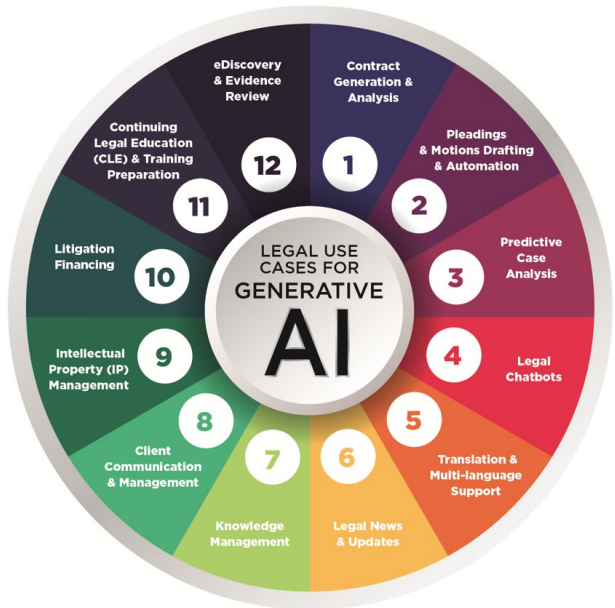
End of Semester Written Examination: 60%

- Assesses your **understanding and application of the concepts and techniques covered throughout the module.**
- May include database design and modelling, advanced SQL, transactions and concurrency, ACID/BASE, CAP, NoSQL and query performance.
- Exact details will be given during exam preparation at the end of the semester.

Continuous Assessment: 40%

- Practical database work completed during the semester.
- Includes **lab activities/tasks and a (small) take-home assignment.**
- Focus is on applying the techniques covered in the module, including PostgreSQL, transactions, programming with databases, permissions, advanced SQL and NoSQL.

GenAI in CMPU3010



This Photo by Unknown Author is licensed under [CC BY](#)

Generative AI tools can be useful when learning and working with databases.

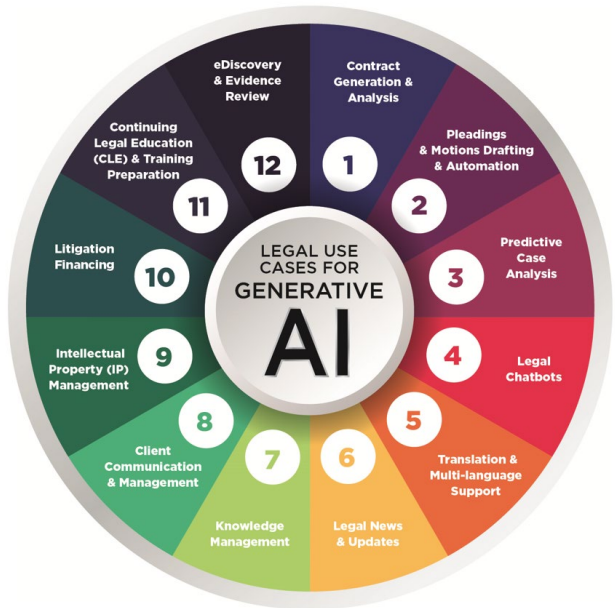
Appropriate uses may include:

- explaining a database concept in a different way;
- helping interpret an error message;
- suggesting approaches to a problem;
- explaining unfamiliar SQL or Python code;
- generating small examples for practice;
- helping with debugging;
- improving the presentation or documentation of your own work.

But: GenAI output is not guaranteed to be **correct, efficient or appropriate.**

You are responsible for everything you submit.

Don't Just Generate — Understand and Verify



This Photo by Unknown Author is licensed under [CC BY](#)

For example, an AI tool might generate:

```
SELECT ...  
FROM ...  
JOIN ...
```

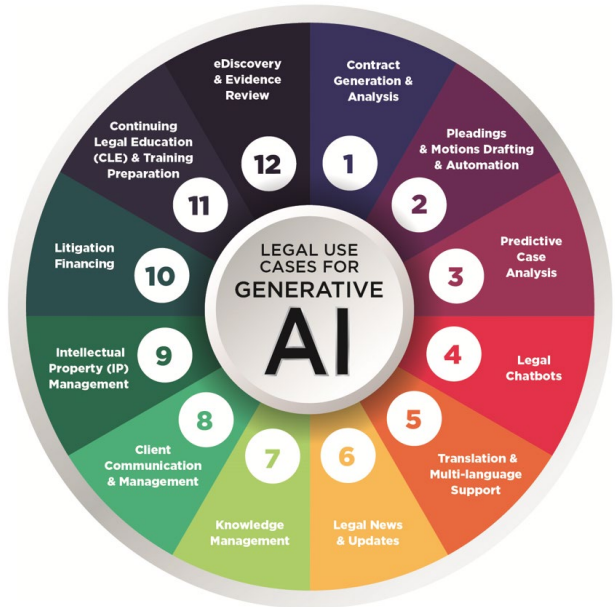
Before using it, you should be able to answer:

- What does this query do?
- Why is this JOIN required?
- Why was this particular approach chosen?
- Does it produce the correct result?
- Does it handle the data correctly?
- Could it be written more efficiently?
- Can you modify it if the requirements change?

The same applies to **SQL, Python, PostgreSQL functions/procedures and MongoDB queries.**

If you cannot explain your solution, you have not demonstrated that you can solve the problem.

GenAI and Assessment



This Photo by Unknown Author is licensed under CC BY

For assessed work:

- Follow the **GenAI requirements specified for each assessment.**
- Do not assume that because a tool can generate a solution, its use is permitted.
- Where GenAI use is permitted, you may be required to **acknowledge or document its use.**
- Submitted code must **run and produce the required results.**
- You may be asked to **explain, modify or demonstrate your own solution.**
- Assessment may include **individual lab tests, quizzes or verification of submitted work.**

Key principle

- Use GenAI to support your learning — not to replace it.

The important progression for CMPU3010 is:

- **Generate/Ask → Understand → Test → Verify → Adapt → Explain**

Reading List

Silva, Rick. (2021), Essential Postgres: Database Development using PostgreSQL, Penguin.

Wickham, Hadley. (2014), Tidy Data, Journal of Statistical Software 59 (2014): 1-23.
<https://vita.had.co.nz/papers/tidy-data.pdf>

MongoDB. What is MongoDB?,
MongoDB.com
<https://www.mongodb.com/docs/manual/>